

JAVASCRIPT & WEB TECHNOLOGIES

Complete Study Notes



- Functions & Events · DOM Traversal · Output Systems · Variables & Arrays
- Date & String Handling · CSS Manipulation · Form Validation
- Advanced JS: OOP · Error Handling · BOM · ES6 Features
- HTML + CSS + JS Integration · Event Handling
- AJAX: Purpose, Advantages, Alternatives
- XML: DTD · Schemas · XSL · XSLT Transformation
- XHTML: Key Concepts & Features
- Keys & Values · Types · Arrays & Objects

■ TABLE OF CONTENTS

UNIT 1: Functions, Events & DOM

- 1.1 JavaScript Functions
- 1.2 Events & Event Handling
- 1.3 DOM Traversal & Manipulation
- 1.4 Output Systems
- 1.5 Variables & Arrays
- 1.6 Date & String Handling

UNIT 2: CSS Manipulation & Form Validation

- 2.1 Manipulating CSS through JavaScript
- 2.2 Form Validation Techniques

UNIT 3: Advanced JavaScript

- 3.1 Error Handling
- 3.2 OOP in JavaScript
- 3.3 Libraries & Frameworks
- 3.4 Browser Object Model (BOM)
- 3.5 ES6 Features

UNIT 4: Combining HTML, CSS & JavaScript

- 4.1 Handling Events & Buttons
- 4.2 Controlling the Browser

UNIT 5: Introduction to AJAX

- 5.1 Purpose & Working of AJAX
- 5.2 Advantages & Disadvantages
- 5.3 AJAX-Based Applications & Alternatives

UNIT 6: Introduction to XML

- 6.1 XML Basics & Key Concepts
- 6.2 DTD & XML Schemas
- 6.3 XSL & XSLT
- 6.4 Transforming XML using XSLT

UNIT 7: XHTML

- 7.1 Key Concepts & Features of XHTML

UNIT 8: Keys, Values, Types, Arrays & Objects

- 8.1 Keys & Values
- 8.2 Types of Values
- 8.3 Arrays in JavaScript

■ 8.4 Objects in JavaScript

UNIT 1: FUNCTIONS, EVENTS & DOM

1.1 JavaScript Functions

A **function** is a reusable block of code that performs a specific task. Functions help avoid code repetition and make programs modular and maintainable.

Types of Functions

Type	Syntax / Description
Function Declaration	<code>function greet(name){ return 'Hello '+name; }</code> — Hoisted; can be called before definition.
Function Expression	<code>const greet = function(name){ ... };</code> — Not hoisted; anonymous function stored in variable.
Arrow Function (ES6)	<code>const greet = (name) => 'Hello '+name;</code> — Shorter syntax; no own 'this'.
IIFE	<code>(function(){ ... })();</code> — Immediately Invoked; creates private scope.
Callback Function	A function passed as argument to another function. Used in async operations.
Higher-Order Function	Takes functions as parameters OR returns a function. e.g. <code>map()</code> , <code>filter()</code> , <code>reduce()</code>

```
// Function Declaration
function add(a, b) {
  return a + b;
}

// Arrow Function (ES6)
const multiply = (a, b) => a * b;

// Default Parameters
function greet(name = 'Guest') {
  return `Hello, ${name}!`; // Template literal
}

// Rest Parameters
```

```
function sum(...nums) {  
  return nums.reduce((acc, n) => acc + n, 0);  
}
```

Scope in Functions

- **Global Scope:** Variables declared outside any function — accessible everywhere.
- **Local/Function Scope:** Variables declared inside a function — only accessible within it.
- **Block Scope (ES6):** `let` and `const` are block-scoped (inside `{}` braces).
- **Closure:** Inner function retains access to outer function's variables even after outer function has returned.

1.2 Events & Event Handling

An **event** is any user action or browser-generated occurrence (click, keypress, page load). JavaScript responds to events through **event handlers** or **event listeners**.

Category	Events	Description
Mouse	click, dblclick, mouseover, mouseout, mousemove	Triggered by mouse interactions
Keyboard	keydown, keyup, keypress	Triggered when keys are pressed/released
Form	submit, change, focus, blur, input	Form and input field interactions
Window	load, resize, scroll, unload	Browser window-level events
Touch	touchstart, touchend, touchmove	Mobile device touch interactions
Drag	dragstart, dragend, drop, dragover	Drag-and-drop interactions

```
// Method 1: Inline HTML (avoid)  
<button onclick="alert('Clicked!')">Click</button>  
  
// Method 2: DOM property assignment  
document.getElementById('btn').onclick = function() {  
  console.log('Button clicked');  
};  
  
// Method 3: addEventListener (BEST PRACTICE)  
document.getElementById('btn').addEventListener('click', function(event) {  
  console.log('Target:', event.target);  
});
```

```
event.preventDefault(); // Stop default action

event.stopPropagation(); // Stop event bubbling

});

// Event Delegation (efficient for many elements)
document.querySelector('#list').addEventListener('click', function(e) {
  if(e.target.tagName === 'LI') { console.log(e.target.textContent); }
});
```

Event Bubbling vs Capturing

- **Bubbling (default):** Event fires on the target element first, then bubbles up to parent elements.
- **Capturing:** Event fires on outermost element first, then drills down to target. Enable with `addEventListener('click', fn, true)`.
- **stopPropagation():** Prevents event from traveling up/down the DOM tree.

1.3 Document Object Model (DOM) Traversal

The **DOM (Document Object Model)** is a tree-structured representation of an HTML document. JavaScript can read, add, change, and delete HTML elements and attributes using the DOM API.

DOM Selection Methods

Method	Returns	Example
<code>getElementById()</code>	Single element	<code>document.getElementById('header')</code>
<code>getElementsByClassName()</code>	HTMLCollection (live)	<code>document.getElementsByClassName('item')</code>
<code>getElementsByTagName()</code>	HTMLCollection (live)	<code>document.getElementsByTagName('p')</code>
<code>querySelector()</code>	First matching element	<code>document.querySelector('.nav > li')</code>
<code>querySelectorAll()</code>	NodeList (static)	<code>document.querySelectorAll('input[type=text]')</code>

DOM Traversal Properties

Property	Description
<code>parentNode / parentElement</code>	Get the parent of current node
<code>childNodes / children</code>	<code>childNodes</code> = all nodes; <code>children</code> = element nodes only
<code>firstChild / firstElementChild</code>	First child node / first element child

lastChild / lastElementChild	Last child node / last element child
nextSibling / nextElementSibling	Next node / next element at same level
previousSibling previousElementSibling	Previous node / previous element

```
// Selecting and modifying elements
const heading = document.querySelector('h1');
heading.textContent = 'New Title'; // Change text
heading.innerHTML = '<em>New Title</em>'; // Change HTML
heading.setAttribute('class', 'highlight'); // Set attribute

// Creating and inserting elements
const newDiv = document.createElement('div');
newDiv.textContent = 'I am new!';
newDiv.classList.add('box');
document.body.appendChild(newDiv); // Add at end
document.body.insertBefore(newDiv, heading); // Add before element

// Removing elements
heading.remove(); // Modern way
heading.parentNode.removeChild(heading); // Old way

// Traversal example
const ul = document.querySelector('ul');
const items = ul.children; // HTMLCollection of li elements
for(let item of items) { console.log(item.textContent); }
```

1.4 Output Systems in JavaScript

Method	Use Case	Example
alert(msg)	Show popup message to user. Blocks execution.	alert('Hello!')
confirm(msg)	OK/Cancel dialog. Returns true/false.	let ok = confirm('Sure?')
prompt(msg)	Input box dialog. Returns string or null.	let name = prompt('Name?')
document.write()	Writes directly to HTML. Overwrites page if called after load.	document.write('Hi')
innerHTML / textContent	Update element content dynamically (preferred).	el.innerHTML = 'Bold'
console.log()	Debug output in browser DevTools console.	console.log(obj)
console.warn()	Warning message in console (yellow).	console.warn('Check!')
console.error()	Error message in console (red).	console.error('Failed!')
console.table()	Display array/object as formatted table.	console.table(users)

1.5 Variables & Arrays

Variable Declarations

Keyword	Scope	Hoisting	Re-assignable
var	Function/Global	Yes (undefined)	Yes
let (ES6)	Block { }	No (TDZ)	Yes
const (ES6)	Block { }	No (TDZ)	No (reference fixed)

JavaScript Arrays

An **array** is an ordered list of values. Arrays in JavaScript are dynamic — they can hold mixed types and resize automatically.

```
// Creating arrays
const fruits = ['apple', 'banana', 'cherry'];
const mixed = [1, 'hello', true, null, {key:'val'}];

// Common array methods
fruits.push('mango'); // Add to end
fruits.pop(); // Remove from end
```

```
fruits.unshift('grape'); // Add to front
fruits.shift(); // Remove from front
fruits.splice(1, 1, 'kiwi'); // Replace at index 1

const nums = [3, 1, 4, 1, 5];
nums.sort((a,b) => a - b); // Sort ascending
const doubled = nums.map(n => n*2);
const evens = nums.filter(n => n%2 === 0);
const total = nums.reduce((acc, n) => acc+n, 0);

// Spread operator
const combined = [...fruits, ...nums];

// Destructuring
const [first, second, ...rest] = fruits;
```

1.6 Date & String Handling

Date Object

```
const now = new Date(); // Current date+time
const d = new Date(2024, 0, 15); // Jan 15, 2024 (month 0-indexed)

now.getFullYear() // 2024
now.getMonth() // 0-11 (add 1 for display)
now.getDate() // 1-31 day of month
now.getDay() // 0-6 (0=Sunday)
now.getHours() // 0-23
now.getTime() // Milliseconds since Jan 1, 1970

// Formatting
now.toLocaleDateString('en-IN') // '15/1/2024'
now.toLocaleString() // Full date+time string
```

String Methods

Method	Description	Example
length	Number of characters	'hello'.length → 5

toUpperCase() toLowerCase()	/	Change case	'Hi'.toLowerCase() → 'hi'
trim() / trimStart() trimEnd()	/	Remove whitespace	' hi '.trim() → 'hi'
split(sep)		Split into array	'a,b'.split(',') → ['a','b']
slice(start, end)		Extract substring	'hello'.slice(1,3) → 'el'
includes(str)		Check if contains substring	'hello'.includes('ell') → true
replace(old, new)		Replace first match	'hello'.replace('l','r') → 'herlo'
replaceAll(old,new)		Replace all matches	'aaa'.replaceAll('a','b') → 'bbb'
indexOf(str)		Position of substring (-1 if not found)	'hello'.indexOf('ll') → 2
padStart(n, ch) padEnd(n,ch)	/	Pad string to length n	'5'.padStart(3,'0') → '005'
Template Literals `\${}`		Embed expressions in strings	`Hi \${name}, you are \${age}!`

UNIT 2: CSS MANIPULATION & FORM VALIDATION

2.1 Manipulating CSS through JavaScript

JavaScript can dynamically change the appearance of HTML elements by modifying their inline styles, CSS classes, or CSS custom properties (variables).

```
const el = document.querySelector('.box');

// 1. Inline style manipulation
el.style.color = 'red';
el.style.backgroundColor = '#1D4ED8';
el.style.fontSize = '18px';
el.style.display = 'none'; // Hide element
el.style.display = 'block'; // Show element

// 2. classList API (BEST PRACTICE)
el.classList.add('active'); // Add class
el.classList.remove('hidden'); // Remove class
el.classList.toggle('dark-mode'); // Toggle on/off
el.classList.contains('active'); // Returns true/false
el.classList.replace('old', 'new'); // Replace class

// 3. Getting computed styles
const styles = window.getComputedStyle(el);
console.log(styles.getPropertyValue('font-size'));

// 4. CSS Custom Properties (variables)
document.documentElement.style.setProperty('--primary-color', '#FF5733');

// 5. Dynamic stylesheet injection
const style = document.createElement('style');
style.textContent = '.box { border: 2px solid gold; }';
document.head.appendChild(style);
```

Common CSS Animation via JS

Use CSS transitions + JS class toggling for smooth animations:

```
el.style.transition = 'all 0.3s ease'; then toggle class — CSS handles the animation.
```

For complex animations, use the **Web Animations API**:

```
el.animate([ {opacity:0}, {opacity:1}], {duration:500})
```

2.2 Form Validation Techniques

Form validation ensures user input meets required criteria before submitting to a server. JavaScript enables **client-side validation** for immediate feedback without a page reload.

Required Validator

```
function validateRequired(value, fieldName) {  
  if (value === null || value === undefined || value.trim() === '') {  
    return { valid: false, msg: fieldName + ' is required.' };  
  }  
  return { valid: true };  
}  
  
// HTML5 built-in: just add required attribute  
<input type='text' required />
```

Length Validator

```
function validateLength(value, min, max) {  
  const len = value.trim().length;  
  if (len < min) return { valid: false, msg: `Min ${min} characters required.` };  
  if (len > max) return { valid: false, msg: `Max ${max} characters allowed.` };  
  return { valid: true };  
}  
  
// HTML5 built-in  
<input type='text' minlength='3' maxlength='20' />
```

Pattern Validator (Regex)

```
function validatePattern(value, pattern, msg) {  
  const regex = new RegExp(pattern);  
  if (!regex.test(value)) return { valid: false, msg };  
  return { valid: true };  
}
```

```
}

// Common patterns
const emailRegex = /^[\\w.-]+@[\\w.-]+\\. [a-z]{2,}$/i;
const phoneRegex = /^[+]?[0-9]{10,13}$/;
const passwordRegex = /^(?=.*[A-Z])(?=.*[0-9])(?=.*[!@#$]).{8,}$/;
const alphanumeric = /^[a-zA-Z0-9]+$/;

// Full form validation example
document.getElementById('form').addEventListener('submit', function(e) {
  e.preventDefault();
  const name = document.getElementById('name').value;
  const email = document.getElementById('email').value;
  let errors = [];
  if (!name.trim()) errors.push('Name is required');
  if (!emailRegex.test(email)) errors.push('Invalid email');
  if (errors.length > 0) { alert(errors.join('\\n')); return; }
  this.submit(); // All valid — submit
});
```

HTML5 Built-in Validators

Attribute	Purpose
required	Field cannot be empty
minlength / maxlength	Minimum/maximum character count
min / max	Minimum/maximum numeric value
pattern	Regex pattern the value must match
type (email/url/number)	Automatic format validation
checkValidity()	JS method — returns true if all constraints pass
setCustomValidity(msg)	Set a custom validation error message

UNIT 3: ADVANCED JAVASCRIPT

3.1 JavaScript Error Handling

Error handling prevents an application from crashing when unexpected conditions occur. JavaScript uses **try-catch-finally** blocks and the **Error** object.

```
try {
  // Code that might throw an error

  let result = riskyFunction();

  JSON.parse('invalid JSON'); // Throws SyntaxError
} catch (error) {
  console.error('Error name:', error.name);
  console.error('Error msg:', error.message);
  console.error('Stack:', error.stack);
} finally {
  // Always runs – cleanup code
  console.log('Execution complete');
}

// Throwing custom errors
function divide(a, b) {
  if (b === 0) throw new Error('Division by zero!');
  return a / b;
}

// Custom Error Classes
class ValidationError extends Error {
  constructor(message, field) {
    super(message);
    this.name = 'ValidationError';
    this.field = field;
  }
}

throw new ValidationError('Invalid email', 'email');
```

Error Type	Cause
SyntaxError	Invalid JavaScript syntax — e.g. missing bracket
ReferenceError	Accessing an undeclared variable
TypeError	Operation on wrong data type — e.g. null.property
RangeError	Value outside allowed range — e.g. new Array(-1)
URIError	Malformed URI in decodeURIComponent()

3.2 JavaScript OOP (Object-Oriented Programming)

JavaScript supports OOP through **prototype-based inheritance**. ES6 introduced **class syntax** as syntactic sugar over the prototype chain.

```
// ES6 Class Syntax

class Animal {
  constructor(name, sound) {
    this.name = name; // Instance property
    this.sound = sound;
  }
  speak() { // Method
    return `${this.name} says ${this.sound}`;
  }
  static create(name, sound) { // Static method
    return new Animal(name, sound);
  }
}

// Inheritance with extends
class Dog extends Animal {
  constructor(name) {
    super(name, 'Woof'); // Call parent constructor
    this.tricks = [];
  }
  learn(trick) { this.tricks.push(trick); }
  speak() { // Method overriding (Polymorphism)
```

```

    return super.speak() + '! Good dog!';
  }
}

const dog = new Dog('Rex');
dog.learn('sit');

console.log(dog instanceof Animal); // true – Inheritance check

// Getters and Setters
class Circle {
  constructor(r) { this._r = r; }
  get area() { return Math.PI * this._r ** 2; }
  set radius(r){ if(r>0) this._r = r; }
}

```

4 Pillars of OOP in JavaScript

- **Encapsulation:** Group data (properties) and behaviour (methods) in a class. Use # prefix for private fields: `#balance = 0`
- **Inheritance:** Child class inherits properties/methods from parent using `extends` and `super()`.
- **Polymorphism:** Child class overrides parent methods. Same method name, different behaviour.
- **Abstraction:** Hide implementation details; expose only essential interface.

3.3 JavaScript Libraries & Frameworks

Name	Type	Purpose
jQuery	Library	DOM manipulation, AJAX, animations with simplified API (\$())
React.js	UI Library	Component-based UI building; Virtual DOM; by Meta/Facebook
Vue.js	Framework	Progressive framework; reactive data binding; easy to learn
Angular	Framework	Full MVC framework by Google; TypeScript-based; enterprise apps
Node.js	Runtime	Run JavaScript on server side; non-blocking I/O; npm ecosystem
Lodash	Utility Library	Helper functions for arrays, objects, strings, numbers
Axios	HTTP Library	Promise-based HTTP client; alternative to fetch(); better error handling

D3.js	Visualization	Data-driven SVG/canvas visualizations; charts and graphs
-------	---------------	--

3.4 Browser Object Model (BOM)

The **BOM** provides JavaScript access to the browser itself (outside the document). The **window** object is the top-level BOM object — all global variables and functions are its properties.

Object	Key Properties / Methods
window	window.innerWidth/Height, window.open(), window.close(), window.scrollTo()
window.location	href, hostname, pathname, search, hash, reload(), assign(url), replace(url)
window.history	back(), forward(), go(n), pushState(), replaceState(), length
window.navigator	userAgent, language, onLine, geolocation, clipboard, platform
window.screen	width, height, availWidth, availHeight, colorDepth
window.localStorage	setItem(k,v), getItem(k), removeItem(k), clear() — persists after tab close
window.sessionStorage	Same API as localStorage — data cleared when tab is closed
Timers	setTimeout(fn, ms), setInterval(fn, ms), clearTimeout(id), clearInterval(id)

3.5 ES6+ Features

Feature	Description
let & const	Block-scoped variable declarations replacing var.
Arrow Functions	Shorter syntax; lexical this: (a, b) => a + b
Template Literals	String interpolation: `Hello \${name}`, multi-line strings
Destructuring	Unpack arrays/objects: const {x, y} = point; const [a, b] = arr;
Spread / Rest (...)	Spread array: [...arr1, ...arr2]. Rest params: function(...args)
Default Parameters	function greet(name = 'Guest') { ... }
Modules (import/export)	export function fn(){ } and import {fn} from './module.js'
Promises & async/await	Handle async operations without callback hell
Classes	Syntactic sugar for prototype-based OOP
Map & Set	Map: key-value pairs (any key type). Set: unique values only.

Symbol	Unique, immutable primitive values — used as object property keys
Optional Chaining (?.)	obj?.prop?.sub — returns undefined instead of error if null
Nullish Coalescing (??)	a ?? b — returns b only if a is null or undefined
for...of loop	Iterate over iterables: for(const item of array)
Object shorthand	const obj = {x, y} instead of {x: x, y: y}

```
// Promises

const fetchData = () => new Promise((resolve, reject) => {
  setTimeout(() => resolve('Data!'), 1000);
});

fetchData().then(data => console.log(data)).catch(err => console.error(err));

// async/await (cleaner way to use Promises)

async function getData() {
  try {
    const response = await fetch('https://api.example.com/data');
    const json = await response.json();
    return json;
  } catch(error) {
    console.error('Fetch failed:', error);
  }
}

// Destructuring + Spread example

const [head, ...tail] = [1, 2, 3, 4]; // head=1, tail=[2,3,4]

const {name, ...rest} = {name:'Ali', age:25, city:'Delhi'};
```

UNIT 4: COMBINING HTML, CSS & JAVASCRIPT

4.1 Handling Events & Buttons

```
<!-- HTML Structure -->

<button id='toggle-btn'>Toggle Dark Mode</button>

<div id='content' class='light-mode'>Hello World</div>

/* CSS */

.light-mode { background: #fff; color: #000; }
.dark-mode { background: #111; color: #fff; transition: all 0.3s; }

// JavaScript

const btn = document.getElementById('toggle-btn');
const content = document.getElementById('content');

btn.addEventListener('click', () => {
  content.classList.toggle('dark-mode');
  content.classList.toggle('light-mode');
  btn.textContent = content.classList.contains('dark-mode')
    ? 'Switch to Light' : 'Toggle Dark Mode';
});

// Multiple buttons – tab system example

document.querySelectorAll('.tab').forEach(tab => {
  tab.addEventListener('click', function() {
    document.querySelector('.tab.active').classList.remove('active');
    this.classList.add('active');
    showTabContent(this.dataset.tab);
  });
});
```

4.2 Controlling the Browser

```
// Navigation control

window.location.href = 'https://example.com'; // Redirect
```

```
window.location.reload(); // Refresh page

window.history.back(); // Go back

window.history.forward(); // Go forward

window.history.go(-2); // Go back 2 pages

// Opening and closing windows

const popup = window.open('page.html', '_blank', 'width=400,height=300');
popup.close();

// Scroll control

window.scrollTo({ top: 0, behavior: 'smooth' }); // Scroll to top

window.scrollBy(0, 200); // Scroll down 200px

// Fullscreen API

document.documentElement.requestFullscreen();

document.exitFullscreen();

// Clipboard API

navigator.clipboard.writeText('Copied text!');

// Geolocation

navigator.geolocation.getCurrentPosition(pos => {
  console.log(pos.coords.latitude, pos.coords.longitude);
});
```

UNIT 5: INTRODUCTION TO AJAX

5.1 Purpose & Working of AJAX

AJAX (Asynchronous JavaScript and XML) is a technique to communicate with the server in the background **without reloading the entire webpage**. It uses the **XMLHttpRequest** object (or modern **Fetch API**) to send/receive data asynchronously.

AJAX Request-Response Flow

1. User triggers an event (button click, form submit).
2. JavaScript creates an XMLHttpRequest (XHR) object.
3. XHR sends an HTTP request to the server asynchronously.
4. Server processes the request and sends back a response (JSON/XML/HTML).
5. JavaScript receives the response and updates only part of the page via DOM.

```
// Traditional XMLHttpRequest

const xhr = new XMLHttpRequest();

xhr.open('GET', 'https://api.example.com/users', true); // true=async

xhr.setRequestHeader('Content-Type', 'application/json');

xhr.onreadystatechange = function() {

  if(xhr.readyState === 4 && xhr.status === 200) {

    const data = JSON.parse(xhr.responseText);

    console.log(data);

  }

};

xhr.send();


// XHR readyState values

// 0=UNSENT 1=OPENED 2=HEADERS_RECEIVED 3=LOADING 4=DONE


// Modern Fetch API (recommended)

fetch('https://api.example.com/users')

.then(response => {

  if(!response.ok) throw new Error('HTTP Error: ' + response.status);

  return response.json();

})
```

```
})  
  
.then(data => {  
  document.getElementById('list').innerHTML =  
  data.map(u => `<li>${u.name}</li>`).join('');  
})  
  
.catch(error => console.error('Fetch error:', error));  
  
// async/await with Fetch (cleanest)  
  
async function loadUsers() {  
  const res = await fetch('/api/users');  
  const data = await res.json();  
  return data;  
}
```

5.2 Advantages & Disadvantages of AJAX

Advantages	Disadvantages
✓ No full page reload — faster UX	✗ JavaScript must be enabled in browser
✓ Reduces server load (partial updates)	✗ Back button & bookmarks may not work properly
✓ Asynchronous — UI remains responsive	✗ SEO issues — crawlers may miss dynamic content
✓ Better user experience (SPAs)	✗ Security: XSS & CSRF vulnerabilities if not handled
✓ Bandwidth efficient	✗ Debugging async code can be complex
✓ Works with REST APIs easily	✗ CORS (Cross-Origin Resource Sharing) restrictions

5.3 Alternatives to AJAX

Technology	Description
Fetch API	Modern promise-based replacement for XHR. Cleaner API, built into browsers.

WebSockets		Full-duplex persistent connection. Server can push data anytime. Ideal for chat, live feeds.
Server-Sent Events (SSE)	Events	One-way server-to-client streaming. Simpler than WebSocket for push notifications.
GraphQL		Query language for APIs. Fetch exactly the data you need — no over/under-fetching.
Long Polling		Client keeps connection open; server responds when data is ready, then client reconnects.
gRPC (web)		High-performance RPC framework by Google. Binary protocol (Protocol Buffers).

UNIT 6: INTRODUCTION TO XML

6.1 XML Basics & Key Concepts

XML (eXtensible Markup Language) is a text-based format for storing and transporting structured data. Unlike HTML, XML tags are user-defined and focus on *data description*, not presentation.

```
<?xml version='1.0' encoding='UTF-8'?>

<library>

  <book id='101'>

    <title>JavaScript: The Good Parts</title>

    <author>Douglas Crockford</author>

    <price currency='USD'>29.99</price>

    <available>true</available>

  </book>

  <book id='102'>

    <title>You Don't Know JS</title>

    <author>Kyle Simpson</author>

    <price currency='USD'>34.99</price>

  </book>

</library>
```

Concept	Description
Well-formed XML	All tags properly nested and closed; case-sensitive; single root element
Elements	Tags that wrap content: <name>Value</name> or empty:
Attributes	Name-value pairs inside opening tag: <book id='101'>
CDATA Section	Wraps text that should not be parsed: <![CDATA[a < b]]>
Namespace	Avoid name conflicts: xmlns:h='http://...' then <h:table>
XML Declaration	<?xml version='1.0' encoding='UTF-8'?> at top of file
Comments	<!-- This is a comment -->
Uses of XML	Config files (Maven, Spring), Web services (SOAP), RSS/Atom feeds, SVG, XHTML, MS Office formats

6.2 DTD & XML Schemas

Document Type Definition (DTD)

A **DTD** defines the legal structure of an XML document — which elements are allowed, their order, and attributes.

```
<!-- Internal DTD (inside XML file) -->
<!DOCTYPE library [
  <!ELEMENT library (book+)> <!-- + means one or more -->
  <!ELEMENT book (title, author, price)>
  <!ELEMENT title (#PCDATA)> <!-- Parsed Character Data -->
  <!ELEMENT author (#PCDATA)>
  <!ELEMENT price (#PCDATA)>
  <!!ATTLIST book id CDATA #REQUIRED> <!-- id attribute required -->
]>

<!-- External DTD -->
<!DOCTYPE library SYSTEM 'library.dtd'>
```

XML Schema (XSD)

XSD is more powerful than DTD — supports data types, namespaces, and is itself written in XML.

```
<xs:schema xmlns:xs='http://www.w3.org/2001/XMLSchema'>
  <xs:element name='book'>
    <xs:complexType>
      <xs:sequence>
        <xs:element name='title' type='xs:string'/>
        <xs:element name='author' type='xs:string'/>
        <xs:element name='price' type='xs:decimal'/>
      </xs:sequence>
      <xs:attribute name='id' type='xs:integer' use='required'/>
    </xs:complexType>
  </xs:element>
</xs:schema>
```

Feature	DTD	XSD
Syntax	Not XML	XML syntax

Data Types	Limited (PCDATA)	Rich: string, integer, date, etc.
Namespaces	Not supported	Fully supported
Extensibility	Limited	Highly extensible
Complexity	Simple	More complex but powerful

6.3 XSL & XSLT

XSL (eXtensible Stylesheet Language) is a family of languages for transforming and rendering XML documents. It has three parts:

- **XSLT (XSL Transformations)**: Transforms XML into HTML, another XML, plain text, or any format.
- **XPath (XML Path Language)**: Navigation language for selecting nodes/values in an XML document.
- **XSL-FO (Formatting Objects)**: Defines formatting for print/PDF output (now largely replaced by CSS).

Key XSLT / XSL Elements

Element	Purpose
<xsl:stylesheet>	Root element of any XSLT document. Declares namespace.
<xsl:template match='/'>	Match rule for a node. '/' matches root of XML document.
<xsl:value-of select='... '>	Outputs the value of a selected node/expression.
<xsl:for-each select='... '>	Loops over a set of selected nodes.
<xsl:if test='... '>	Conditional output — if the test expression is true.
<xsl:choose> / <xsl:when>	Switch-case equivalent for multiple conditions.
<xsl:sort select='... '>	Sort nodes before output.
<xsl:apply-templates>	Apply matching templates to child nodes.
<xsl:output method='... '>	Specify output format: html, xml, or text.
<xsl:variable name='... '>	Declare a variable in XSLT.
<xsl:param name='... '>	Declare a parameter (can be passed from outside).

6.4 Transforming XML using XSLT — Complete Example

```
<!-- students.xml -->

<?xml version='1.0' encoding='UTF-8'?>

<?xml-stylesheet type='text/xsl' href='students.xsl'?>

<students>
```

```
<student roll='101'>
  <name>Aarav Sharma</name>
  <marks>88</marks>
</student>
<student roll='102'>
  <name>Priya Mehta</name>
  <marks>94</marks>
</student>
</students>
```

```
<!-- students.xsl -->
<?xml version='1.0' encoding='UTF-8'?>
<xsl:stylesheet version='1.0' xmlns:xsl='http://www.w3.org/1999/XSL/Transform'>
  <xsl:output method='html' />

  <xsl:template match='/'>
    <html><body>
      <h2>Student List</h2>
      <table border='1'>
        <tr><th>Roll</th><th>Name</th><th>Marks</th></tr>
        <xsl:for-each select='students/student'>
          <xsl:sort select='marks' order='descending' />
          <tr>
            <td><xsl:value-of select='@roll' /></td>
            <td><xsl:value-of select='name' /></td>
            <td>
              <xsl:if test='marks > 90'>
                <b><xsl:value-of select='marks' /></b>
              </xsl:if>
              <xsl:if test='marks <= 90'>
                <xsl:value-of select='marks' />
              </xsl:if>
            </td>
          </tr>
        </xsl:for-each>
      </table>
    </body>
  </template>
</xsl:stylesheet>
```

```
</xsl:for-each>
</table>
</body></html>
</xsl:template>
</xsl:stylesheet>
```

UNIT 7: INTRODUCTION TO XHTML

7.1 XHTML — Key Concepts & Features

XHTML (eXtensible HyperText Markup Language) is a stricter, XML-based version of HTML. It was developed to make HTML conform to XML rules, enabling better interoperability and processing by XML tools.

XHTML vs HTML — Key Differences

Rule	HTML	XHTML
Tag case	Case-insensitive (<P> OK)	Lowercase only (<p>)
Closing tags	Optional for some ()	Mandatory ()
Attribute values	Can be unquoted (value)	Must be quoted ('value')
Empty elements	<hr>, 	,
Attribute minimization	Allowed: checked, disabled	Must be: checked='checked'
Root element	<html>	Must declare XML namespace
DOCTYPE	Optional	Mandatory
Nesting	Often forgiven by browsers	Must be perfectly nested
Error handling	Browser tries to fix errors	Parser stops on first error

```
<!-- Correct XHTML 1.0 Strict Document -->

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Strict//EN"
'http://www.w3.org/TR/xhtml1/DTD/xhtml1-strict.dtd'>

<html xmlns='http://www.w3.org/1999/xhtml' xml:lang='en' lang='en'>
<head>

<meta http-equiv='Content-Type' content='text/html; charset=UTF-8' />
<title>XHTML Example</title>
</head>
<body>

<h1>Hello XHTML World</h1>

<p>All tags must be in <em>lowercase</em> and properly closed.</p>
<br />

<img src='logo.png' alt='Logo' />

<input type='checkbox' checked='checked' />
```

```
</body>
```

```
</html>
```

XHTML Versions

Version	Description
XHTML 1.0 Strict	Cleanest version — no deprecated elements, no frames, no inline styles
XHTML 1.0 Transitional	Allows deprecated HTML 4 elements (like , <center>) for backwards compatibility
XHTML 1.0 Frameset	Allows use of <frameset> — for framed web pages
XHTML 1.1	Modular design — can include only needed modules. Based on strict.
XHTML 5	HTML5 re-serialized as XML — served as application/xhtml+xml

Why XHTML Matters

- Ensures documents are well-formed and processable by XML tools.
- Better integration with XML-based technologies (XSLT, XPath, SVG, MathML).
- Important for understanding the evolution from HTML4 → XHTML → HTML5.
- Used in some enterprise and scientific publishing workflows.

UNIT 8: KEYS, VALUES, TYPES, ARRAYS & OBJECTS

8.1 Keys & Values

In JavaScript, objects store data as **key-value pairs**. Keys are property names (strings or Symbols); values can be any JavaScript value.

```
const person = {  
  name: 'Aarav', // string key, string value  
  age: 25, // string key, number value  
  active: true, // string key, boolean value  
  address: { // string key, object value (nested)  
    city: 'Delhi',  
    pin: '110001'  
  },  
  greet: function() { // string key, function value (method)  
    return `Hi, I'm ${this.name}`;  
  }  
};  
  
// Accessing values  
person.name // Dot notation → 'Aarav'  
person['age'] // Bracket notation → 25  
const key = 'active';  
person[key] // Dynamic key access → true  
  
// Getting all keys, values, entries  
Object.keys(person) // ['name', 'age', 'active', 'address', 'greet']  
Object.values(person) // ['Aarav', 25, true, {...}, fn]  
Object.entries(person) // [['name', 'Aarav'], ['age', 25], ...]  
  
// Adding / Updating / Deleting  
person.email = 'a@b.com'; // Add new key  
person.age = 26; // Update existing  
delete person.active; // Delete a key
```

8.2 Types of Values in JavaScript

JavaScript has **8 data types** — 7 primitive + 1 complex:

Type	Category	Example	Notes
Number	Primitive	42, 3.14, NaN, Infinity	64-bit float; NaN = Not a Number
BigInt	Primitive	9007199254740991n	For integers beyond Number.MAX_SAFE_INTEGER
String	Primitive	'hello', "world", `\${x}`	Immutable sequence of UTF-16 characters
Boolean	Primitive	true, false	Logical values; used in conditions
undefined	Primitive	let x; // x is undefined	Variable declared but not assigned
null	Primitive	let x = null;	Intentional absence of value (typeof null = 'object' — bug)
Symbol	Primitive	Symbol('id')	Unique, immutable; used as object keys
Object	Complex/Reference	{}, [], function(){}	Arrays, functions, dates are all Objects

```
// Type checking
typeof 42 // 'number'
typeof 'hello' // 'string'
typeof true // 'boolean'
typeof undefined // 'undefined'
typeof null // 'object' ← known JS bug
typeof {} // 'object'
typeof [] // 'object' ← arrays are objects!
typeof function(){} // 'function'
Array.isArray([]) // true ← correct way to check arrays

// Type conversion
Number('42') // 42
String(42) // '42'
Boolean(0) // false (falsy values: 0, '', null, undefined, NaN)
parseInt('42px') // 42
parseFloat('3.14x') // 3.14
```

```
+'42' // 42 (unary plus trick)
```

8.3 Arrays in Depth

Arrays are **ordered, zero-indexed lists**. They are objects with numeric keys and a special **length** property.

Method	Returns	Description
push(val) / pop()	new length / popped value	Add/remove at end
unshift(val) / shift()	new length / removed value	Add/remove at beginning
splice(i, n, ...items)	Array of removed items	Remove n items at index i and optionally insert
slice(start, end)	New array (copy)	Extract a portion without modifying original
concat(...arrays)	New array	Merge arrays; original unchanged
indexOf(val) / lastIndexOf(val)	Index or -1	Find position of value
find(fn) / findIndex(fn)	Element / index	Find first element matching condition
includes(val)	boolean	Check if value exists in array
map(fn)	New array	Transform each element; return new array
filter(fn)	New array	Keep elements where fn returns true
reduce(fn, init)	Single value	Accumulate array to single value
forEach(fn)	undefined	Iterate; no return value
sort(fn?)	Sorted array (in-place)	Sort; provide comparator for numbers
flat(depth) / flatMap(fn)	New array	Flatten nested arrays; flatMap = map + flat
every(fn) / some(fn)	boolean	every: all match? some: any match?
Array.from(iterable)	New array	Create array from string, Set, Map, NodeList
Array.of(...vals)	New array	Create array from arguments

8.4 Objects in Depth

```
// Object creation methods

const obj1 = { x: 1 }; // Object literal
```

```
const obj2 = new Object({ x: 1 }); // Constructor
const obj3 = Object.create(obj1); // Prototype-based

// Object utility methods
Object.assign(target, source); // Copy properties
const clone = { ...original }; // Spread clone
Object.freeze(obj); // Prevent modifications
Object.seal(obj); // Prevent add/delete
Object.keys(obj); Object.values(obj); // Keys / Values arrays
Object.entries(obj); // [[key,val], ...]
Object.fromEntries([['a',1],['b',2]]); // { a:1, b:2 }
'key' in obj; // Property existence check
obj.hasOwnProperty('key'); // Own (not inherited) prop

// Computed property names
const prefix = 'user';
const profile = { [`_${prefix}_id`]: 42, [`_${prefix}_name`]: 'Ali' };
// { user_id: 42, user_name: 'Ali' }

// Destructuring with rename and defaults
const { name: userName = 'Anonymous', age = 0 } = person;

// Prototype chain
const animal = { breathes: true };
const dog = Object.create(animal); // dog's prototype = animal
dog.breathes; // true (inherited from prototype chain)
Object.getPrototypeOf(dog) === animal; // true
```

■ QUICK REVISION — ALL TOPICS AT A GLANCE

Topic	Key Points
Functions	Declaration/Expression/Arrow/IIFE. Scope: var=function, let/const=block. Closures retain outer scope.
Events	addEventListener is best practice. Bubbling (default): child→parent. event.preventDefault() / stopPropagation().
DOM	querySelector/querySelectorAll. Traversal: parentNode, children, nextElementSibling. createElement+appendChild.
Output	alert/confirm/prompt (dialogs). innerHTML/textContent (DOM). console.log/warn/error/table (DevTools).
Variables	var=hoisted+function-scope. let=block-scope. const=block-scope+no-reassign.
Arrays	Ordered, zero-indexed objects. Key methods: map/filter/reduce/find/sort/splice/slice/flat.
Strings	Immutable. Key: split/slice/includes/replace/trim/padStart. Template literals: `\${expr}`.
CSS via JS	el.style.prop. classList.add/remove/toggle/contains. getComputedStyle(). setProperty() for vars.
Form Validation	Required: trim()=== check. Length: value.length vs min/max. Pattern: RegExp.test(). HTML5: required/pattern attrs.
Error Handling	try-catch-finally. Error types: SyntaxError/TypeError/ReferenceError/RangeError. Custom error classes.
OOP	class, constructor, extends, super. 4 pillars: Encapsulation/Inheritance/Polymorphism/Abstraction. # for private.
BOM	window.location (redirect/reload). history (back/forward). navigator (userAgent, geolocation). localStorage/sessionStorage.
ES6	Arrow fn, template literals, destructuring, spread/rest, classes, modules, Promises, async/await, Map/Set.
AJAX	Async HTTP without page reload. XHR or Fetch API. Alternatives: WebSocket, SSE, GraphQL.
XML	Self-describing data format. Well-formed rules: lowercase, closed, nested. DTD/XSD for validation.

XSLT	Transform XML to HTML/text. Key: template match, for-each, value-of, if, sort, choose.
XHTML	Strict XML-based HTML. Rules: lowercase tags, all closed, quoted attrs, no minimization, mandatory DOCTYPE.
Types	7 primitives: number/bigint/string/boolean/null/undefined/symbol. 1 complex: object. typeof for checking.
Objects	Key-value pairs. Access: dot or bracket. Object.keys/values/entries/assign/freeze. Prototype chain.